

Recombination in a Self-Modifying Network Search: Building Blocks, and the Weight Inheritance That Crossover Needs

Vasili Gavrilov*

2026

Abstract

Ordinarily a neural network is handed its shape before it learns; the system here is handed only the data and a target error and evolves the rest, its depth and width, whether each layer is dense or convolutional, and its own learning rate, momentum, and activation. Onto that mutation-driven search we add *crossover*, and ask, across four small controlled experiments, when recombining two parents beats mutating one. The answer is a single rule. On concatenated deceptive trap functions, where the solution splits into independent blocks, our crossover beats mutation decisively and its advantage grows with the number of blocks. On the neural architecture search it does nothing: an architecture's only heritable trait is its trainability, a single entangled quantity with no separable sub-parts to recombine. Give the genome the missing piece, *weight inheritance*, so a candidate carries its trained modules across generations, and crossover wins again by a wide margin. Crossover helps exactly insofar as the genome carries separable, transferable building blocks. On a real NAS benchmark (NAS-Bench-201) that rule is harder to see: the cell space is small and crossover's benefit is tiny and not predicted by separability, a limit we report honestly. **None of these mechanisms is new:** weight inheritance is the Lamarckian evolution of LEMONADE [14], modular crossover is that of He et al. [18], and random search as a strong baseline is Li and Talwalkar [11]. The contribution is a single minimal dependency-free implementation that reproduces and unifies them under one through-line with a trap-function positive control, every figure regenerable from one command. It is a reproduction and synthesis, offered as such.

1 Introduction

Neural architecture search (NAS) automates the choice of a network's shape; methods differ mainly in the search strategy, reinforcement learning [8], gradient relaxation [10], Bayesian optimization, random search, and evolution [12]. Ours is evolutionary: the topology is evolved while each candidate's weights are trained by back-propagation [9]. The ordinary signature **predict(topology, activation, ...)** becomes **self_modify_predict(data, error)**, and the rest is discovered. The engine grew out of the author's 1997 seminar thesis, which derived the multi-layer delta rule the training still uses [2].

A recurring result is that random search is hard to beat [7, 11]. We ask whether *recombination* beats mutation at this search, and reach a clean rule: crossover needs building blocks that carry a solution; standard NAS does not provide them; weight inheritance does. The self-adaptive machinery is standard in the evolution-strategy literature [3].

*Independent researcher. Physics and computer science by education; a professional software engineer with a background in numerical optimization and machine learning, including rule engines and expert systems. Reference implementation: <https://github.com/Anode1/SMBPANN>.

Table 1: Crossover versus mutation on concatenated deceptive traps ($K = 4$, population 1000, tournament 3, 200 seeds). Crossover’s advantage grows with the number of blocks; by sixteen, mutation mostly fails while crossover still solves.

blocks M	mutation-only	crossover + mutation	speedup
4	9.2 gens, 200/200	5.1 gens, 200/200	$1.8\times$
8	70.8 gens, 200/200	14.2 gens, 200/200	$5\times$
12	257 gens, 185/200	22.6 gens, 200/200	$11\times$
16	363 gens, 28/200	42.3 gens, 199/200	mutation fails

2 Method

A network is a feed-forward stack of dense and convolutional layers; g is a per-network hidden activation (sigmoid, tanh, or ReLU) chosen by the genome, the output a dense sigmoid. A convolutional layer applies F shared kernels of width K , so its width is derived, $d_\ell = F(d_{\ell-1} - K + 1)$, the local weight-sharing bias LeCun introduced [6], here a move the search can take. Weights train by back-propagation with momentum (the 1997 thesis’s delta rule [2]), initialized in $[-2.4/f_i, 2.4/f_i]$; all arithmetic is 32-bit float, all randomness a seeded xorshift. A *genome* carries the topology, its training hyper-parameters, and a self-adaptive mutation rate [3]; one mutation perturbs, flips, adds, or removes a layer. The search keeps a population, retains the best few, and refills by reproduction, against a matched random-search control. Optional *crossover* (evolve -X) builds a child by a one-point splice of two parents’ hidden-layer lists plus a blend of hyper-parameters; with -X 0 the run is byte-identical to the mutation-only search.

3 Experiments

3.1 The operator works: a building-block control

We first confirm the operator does what crossover should, on concatenated deceptive trap functions: M blocks of K bits, a block scoring K only if all its bits are one and otherwise $K - 1$ minus its number of ones, a deceptive gradient toward all-zero. The optimum is reachable only by assembling solved blocks. We race the same GA with one-point crossover against mutation-only over 200 seeds (make `bbtest`).

The operator is sound: when separable building blocks exist, crossover finds and combines them, and its advantage scales with the amount of structure (Table 1).

3.2 On the architecture search, crossover does not help, and why

We run the same on/off comparison on the neural search: for each seed a fresh synthetic task ($d = 8$, a random-projection sine label), the search run twice from an identical initial population, mutation-only and with crossover, to a target error. The two are indistinguishable; adding recombination does nothing, and not because the operator is weak (Table 1). Two measurements say why. Architecture alone does not solve the task, every topology scores at or below chance (0.25–0.59 test-MSE) with default hyper-parameters; the *hyper-parameters* do the work, and brittly, holding a 8,4,1 topology and varying only them swings the error from 0.09 to 0.65, while with good hyper-parameters fixed the topology matters narrowly and non-monotonically (8,4,1 $\rightarrow$ 0.09, 8,8,1 $\rightarrow$ 0.37). A parallel-branch probe (make `modnas`, gradient-checked) adds that even a purpose-built modular architecture does not decompose, because neural capacity is fungible: one wide branch does what several specialised ones would. So the architecture has a heritable trait, its trainability, but that trait is a single entangled quantity with no independent sub-parts; there are no building blocks to recombine, even though there is something to inherit.

Table 2: Weight-inheriting crossover versus mutation on the modular task (population 30, 300 fine-tune steps/generation, 60 seeds, cap 120 generations). Crossover solves every task in about twelve generations whatever the count; mutation is several times slower and, as components accumulate, often fails.

components G	mutation (re-init)	crossover (inherit)
2	60/60, 31.0 gens	60/60, 11.4 gens
3	20/60 , 47.9 gens	60/60, 12.1 gens
4	59/60, 84.6 gens	60/60, 12.6 gens
5	59/60, 39.4 gens	60/60, 12.5 gens

3.3 Weight inheritance restores the building blocks

The fix is to let the genome carry the learned state, not only the shape. We test it in a modular setting (`make modevo`, gradient-checked): a candidate is G independent sub-networks, one per output component, each a hard sine of its own input group so that training from a random start succeeds only sometimes. A candidate *carries its sub-networks’ trained weights across generations*; crossover recombines whole solved sub-networks from two parents, while mutation must re-roll the initialization lottery for a stuck component within one lineage.

With weights inherited (Table 2) crossover wins again, solving 60/60 in roughly twelve generations independent of the count while mutation is 2.7 to 6.7 $\times$ slower and unreliable. The recombination that did nothing to an architecture-only genome is decisive once the genome carries the solution, exactly as on the trap bits.

Wired through the full engine (`evolve -w`), a checkpoint of each candidate’s trained weights crosses the worker boundary and a child warm-starts from its parent over the layers a mutation left unchanged. A 300-seed-per-cell sweep (`scripts/inherit_sweep.sh`) shows inheritance lowers the search’s final error at equal compute in every cell (exact sign test $p < 10^{-3}$), by a relative 21% at a 20-epoch budget down to 6% at 160: warm-starting helps most when training is scarce and there is discarded training to reclaim, and fades as candidates converge on their own. With inheritance off the run is byte-identical to before, and the random control, having no lineage, is identical in both arms.

3.4 Where architecture is decisive, the search finds convolution

Our tasks make architecture fungible; the opposite regime exists. On a translation-invariant image task (`make conv2d`, a 3×3 diagonal motif at a random position in a noisy 8×8 image, label its orientation) a 2D convolution that detects the motif anywhere reaches 0.1% test error from 2000 images while a same-size dense network is near chance (39%), catching up only with an order of magnitude more data. Here the right structure carries the solution. Giving the search a native 2D convolution front-end (F filters of $K \times K$ with global max-pool, `conv2f`, gradient-checked) and running it on this task (`gentask -I 8`), over 40 seeds it discovers the convolution in *every one*, reaching a median test-MSE of 2×10^{-7} , five orders of magnitude below the dense architectures it can equally build. On the fungible tasks it never preferred convolution; given a task where structure is decisive and reachable, the search finds the local weight-sharing bias LeCun designed by hand.

3.5 On a real benchmark, the rule is harder to see

The trap control is clean by construction; a real NAS space is not. On NAS-Bench-201 [11] (its topology cell, 15,625 architectures with precomputed CIFAR-10/100 and ImageNet16 accuracies, so fitness is a table lookup, `make nasxover`) crossover yields only a small early-convergence gain

over mutation (at most a few tenths of a percent accuracy, and none at a generous budget, both operators nearly reaching the optimum because the cell has only six genes). More tellingly, additive separability does *not* predict crossover’s benefit: over 120 sub-spaces per dataset (fixed size, varying which operations are allowed) the correlation between the additive-model R^2 and crossover’s benefit is null ($r = -0.06, -0.03, -0.11$, none significant). The building-block rule is clean on designed problems but does not straightforwardly transfer to real NAS cell spaces, which are small and do not decompose in a way separability captures. We report this rather than tune a predictor until one correlates.

4 Relation to prior work

None of the mechanisms here is new. Random search as a strong NAS baseline is [7, 11]; mutation-only evolutionary NAS with aging is [9]; weight inheritance, a child warm-started from its trained parent, is exactly the Lamarckian inheritance of LEMONADE [14], built on the function-preserving transfer of Net2Net [15] and Network Morphism [16] and related to ENAS parameter sharing [17]; crossover that inherits trained modules is He et al.’s modular inheritable crossover [18]; and the alignment that makes topology crossover meaningful is NEAT’s historical marking [13]. This work contributes no method beyond these. It offers a single, minimal, dependency-free implementation that reproduces and unifies them under one through-line, recombination helps insofar as the genome carries separable transferable building blocks, with a trap-function positive control (Table 1) isolating that condition and every figure regenerable from one command. It is a reproduction and synthesis, not a discovery.

5 Discussion and conclusion

The experiments compose into one rule: crossover is exactly as useful as the genome is a carrier of separable building blocks. An architecture genome carries only trainability, a single entangled quantity, so recombination has nothing to combine, which is a structural reason random search is strong in NAS and the field [9] drops crossover; weight inheritance gives the genome a second, decomposable thing to carry, the trained solution, and the advantage returns (Table 2). The same search, given a decisive and reachable structure, rediscovers 2D convolution on an image task unaided. On a real tabular benchmark the rule is muted, its cell space too small and too entangled for separability to bite. **Limitations:** scale is small and tasks are mostly synthetic, mean squared error stands in for classification error, and the building-block predictor does not carry to real NAS (Section 3.5), all reported openly. The natural next step is the same comparison on larger, hierarchical search spaces where recombination has more room to matter.

Reproducibility

Every result regenerates from [19] with one command. The engine builds with `make` (C99, no dependencies) into `smbpann`, `evolve` (with `-X` crossover and `-w` weight inheritance), and `gentask`; `make ut` runs the unit and gradient-check suite under AddressSanitizer and UBSan. Standalone probes: `make bbtest` (Table 1), `make modnas`, `make modevo` (Table 2), `make conv2d`, and `make nasxover` (Section 3.5). Search A/Bs are `scripts/xover_ab.sh`, `scripts/inherit_ab.sh`, and `scripts/inherit_sweep.sh`. Each run uses a fixed 32-bit xorshift generator, so the numbers reproduce exactly.

References

- [1] D. E. Rumelhart, G. E. Hinton, R. J. Williams. Learning representations by back-propagating errors. *Nature* 323, 1986.
- [2] V. Gavrilov. *Backpropagation Feed-Forward Neural Networks*. Seminar in Artificial Intelligence, Open University of Israel, 1997 (revised digital edition, Zenodo 2026). <https://doi.org/10.5281/zenodo.20450526>
- [3] I. Rechenberg. *Evolutionsstrategie*. 1973; H.-P. Schwefel, *Numerical Optimization of Computer Models*, 1981.
- [4] J. H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- [5] K. Deb, D. E. Goldberg. Analyzing Deception in Trap Functions. *Foundations of Genetic Algorithms* 2, 1993.
- [6] Y. LeCun, B. Boser, J. S. Denker, et al. Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation* 1(4), 1989.
- [7] J. Bergstra, Y. Bengio. Random Search for Hyper-Parameter Optimization. *JMLR* 13, 2012.
- [8] B. Zoph, Q. V. Le. Neural Architecture Search with Reinforcement Learning. *ICLR* 2017.
- [9] E. Real, A. Aggarwal, Y. Huang, Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI* 2019.
- [10] H. Liu, K. Simonyan, Y. Yang. DARTS: Differentiable Architecture Search. *ICLR* 2019.
- [11] L. Li, A. Talwalkar. Random Search and Reproducibility for Neural Architecture Search. *UAI* 2019 (see also NAS-Bench-201, Dong and Yang, *ICLR* 2020).
- [12] T. Elsken, J. H. Metzen, F. Hutter. Neural Architecture Search: A Survey. *JMLR* 20, 2019.
- [13] K. O. Stanley, R. Miikkulainen. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10(2), 2002.
- [14] T. Elsken, J. H. Metzen, F. Hutter. Efficient Multi-objective Neural Architecture Search via Lamarckian Evolution. *ICLR* 2019.
- [15] T. Chen, I. Goodfellow, J. Shlens. Net2Net: Accelerating Learning via Knowledge Transfer. *ICLR* 2016.
- [16] T. Wei, C. Wang, Y. Rui, C. W. Chen. Network Morphism. *ICML* 2016.
- [17] H. Pham, M. Y. Guan, B. Zoph, Q. V. Le, J. Dean. Efficient Neural Architecture Search via Parameter Sharing. *ICML* 2018.
- [18] C. He, H. Tan, S. Huang, R. Cheng. Efficient evolutionary neural architecture search by modular inheritable crossover. *Swarm and Evolutionary Computation* 64, 2021.
- [19] V. Gavrilov. SMBPANN: a self-modifying backpropagation feed-forward neural network. Source code, <https://github.com/Anode1/SMBPANN>.